

Topic 1: Review of C++ Basics and Introduction to OOP

1.1 Why OOP?

- Procedural programming organises code as a sequence of functions -- works for small programs but becomes hard to manage as size grows
- OOP (Object-Oriented Programming) organises code around objects that bundle data and the functions that operate on that data
- Benefits: code reuse through inheritance, easier maintenance, better modelling of real-world entities
- C++ supports both procedural and OOP styles -- you choose what suits the problem
- The four pillars of OOP: Encapsulation, Abstraction, Inheritance, Polymorphism

1.2 Quick Review: C++ Fundamentals

- Variables: `int age = 20; double gpa = 3.8; char grade = 'A'; string name = "Ali";`
- Functions: declared with return type, name, and parameter list -- must be defined before use or forward-declared
- References: `int& ref = x;` -- an alias for an existing variable, no copy made
- Pointers: `int* p = &x;` -- stores address; `*p` dereferences to the value
- Memory regions: stack (local variables, automatic cleanup) and heap (dynamic allocation with `new/delete`)

```
#include <iostream>
using namespace std;

int square(int n) { return n * n; }

int main() {
    int x = 5;
    int& r = x;    // reference
    int* p = &x;   // pointer
    cout << square(x) << endl;
    return 0;
}
```

1.3 struct vs class -- First Look

- struct groups related variables (and optionally functions) under one name
- In C++, struct and class are nearly identical -- the only difference is default access: struct members are public, class members are private

- Use struct for simple data containers; use class when you need encapsulation and behaviour

```
struct Point {  
    double x, y;    // public by default  
};  
  
class Circle {  
    double radius;  // private by default  
public:  
    void setRadius(double r) { radius = r; }  
    double area() { return 3.14159 * radius * radius; }  
};
```

1.4 Object-Oriented Thinking

- Object: a real-world entity with state (data) and behaviour (functions)
 - Class: a blueprint that defines what data and functions an object will have
 - Example: a BankAccount class has state (balance, owner) and behaviour (deposit, withdraw)
 - Each object created from a class is called an instance
 - Multiple objects of the same class exist independently -- each has its own copy of data members
-